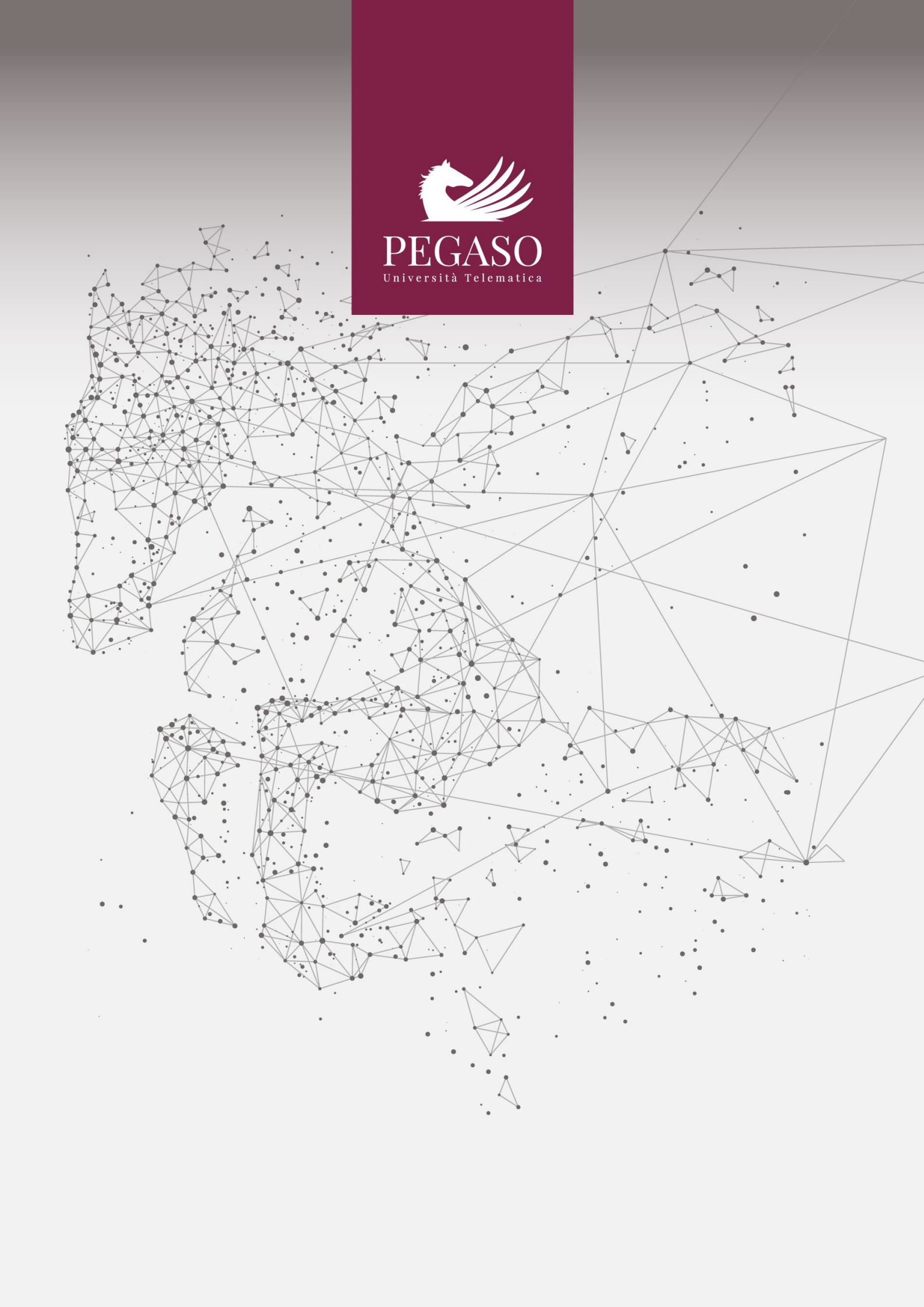

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

# Indice

**1. INTRODUZIONE.....3**

**2. CONCETTI DEL SOFTWARE.....4**

**3. MODELLAZIONE OBJECT ORIENTED.....5**

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

# 1. Introduzione

Perchè modellare il Software? Il Software è già un'astrazione: perché modellarlo? Il Software cresce continuamente e un singolo programmatore non può gestire interamente una simile quantità di codice. Il codice spesso non è direttamente comprensibile dagli sviluppatori che non hanno preso parte allo sviluppo. Abbiamo bisogno di una rappresentazione più semplice per sistemi complessi e la modellazione è un mezzo per gestire la complessità. Diamo alcune definizioni che ci aiuteranno a comprendere meglio i concetti della lezione.

Un **modello** è un'astrazione che descrive un sistema o un sottosistema.

Una **vista** mostra alcuni aspetti selezionati di un modello.

Una **notazione** è un insieme di regole grafiche o testuali per rappresentare viste.

Viste e modelli di un singolo sistema possono sovrapporsi le une agli altri.

Un **fenomeno** è un oggetto nell'universo di dominio così come lo percepiamo, ad esempio: la lezione che state seguendo, il mio orologio.

Un **concetto:** Descrive le proprietà comuni dei fenomeni, ad esempio: le lezioni di Ingegneria del Software, gli smartwatch. Un concetto è rappresentato tramite una tripla: il suo Nome lo distingue dagli altri concetti, il suo Scopo rappresenta le proprietà che determinano se un fenomeno è un membro di un concetto, i suoi Membri sono i fenomeni che sono parte del concetto.

**Un'astrazione** è una classificazione di fenomeni in concetti

La **modellazione** si definisce come lo sviluppo di astrazioni per rispondere a domande specifiche su un insieme di fenomeni rilevanti, ignorando i dettagli irrilevanti.

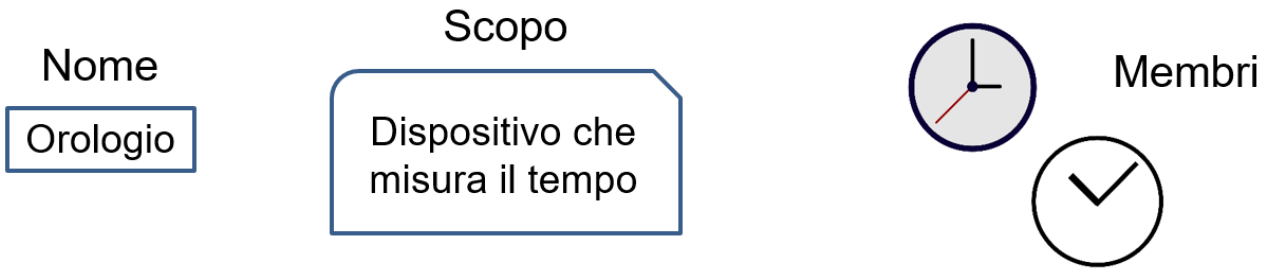



Figura 1 - esempio di concetto

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

# 2. Concetti del software

**I tipi e le istanze.**

**Tipo** è un'astrazione nel contesto di un linguaggio di programmazione**.** Per esempio nome: int, scopo: numero intero, membri: 0, -1, 1, 2, -2, . . .

**Istanza**: è un membro di uno specifico tipo**.**

Il tipo di una variabile rappresenta tutte le possibili istanze che una variabile può assumere**.** La relazione tra "tipo" e "istanza" è simile alla relazione tra "concetto" e "fenomeno". Il **Tipo di dato astratto** è un tipo speciale la cui implementazione è nascosta al resto del sistema.

**Classe** è un'astrazione nel contesto della modellazione e dei linguaggi di programmazione Object-Oriented. Come un tipo di dato astratto, una classe incapsula sia lo stato (attributi), sia il comportamento (metodi). Diversamente dai tipi di dati astratti, le classi possono essere definite in termini di altre classi usando l'ereditarietà. Le classi di eventi rappresentano una tipologia di eventi per i quali il sistema ha una risposta comune. Un evento è una istanza di una classe di eventi. L'invio di messaggi è il meccanismo attraverso il quale l'oggetto mittente richiede l'esecuzione di una operazione da parte dell'oggetto destinatario.

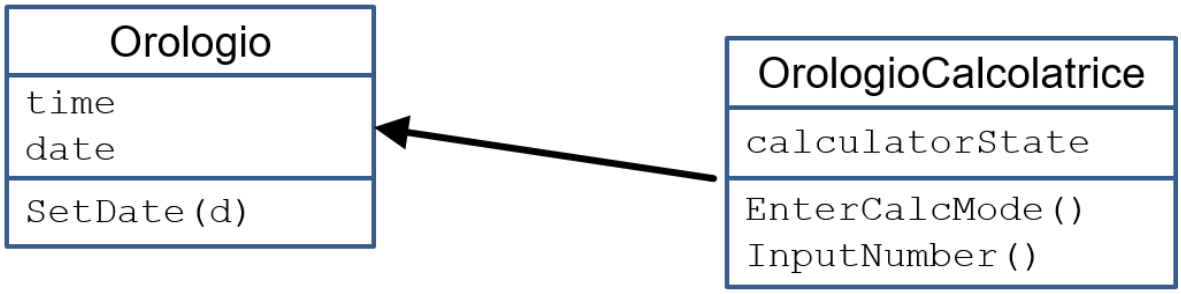



Figura 2 - Classe UML

**Il Dominio di Applicazione** rappresenta tutti gli aspetti del problema dell'utente finale. Coincide con la fase di Analisi dei Requisiti). Viene valutato l'ambiente nel quale il sistema opera, chi sono gli utenti, quali i processi di business, dominio utente e dominio clienti.

Il **Dominio di Soluzione** (System Design, Object Design) bisogna tener conto delle tecnologie disponibili per costruire il sistema.

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

# 3. Modellazione Object Oriented

La **modellazione orientata agli oggetti** è un approccio alla modellazione di sistema software che viene utilizzato all'inizio del ciclo di vita del software quando si utilizza un approccio orientato agli oggetti.

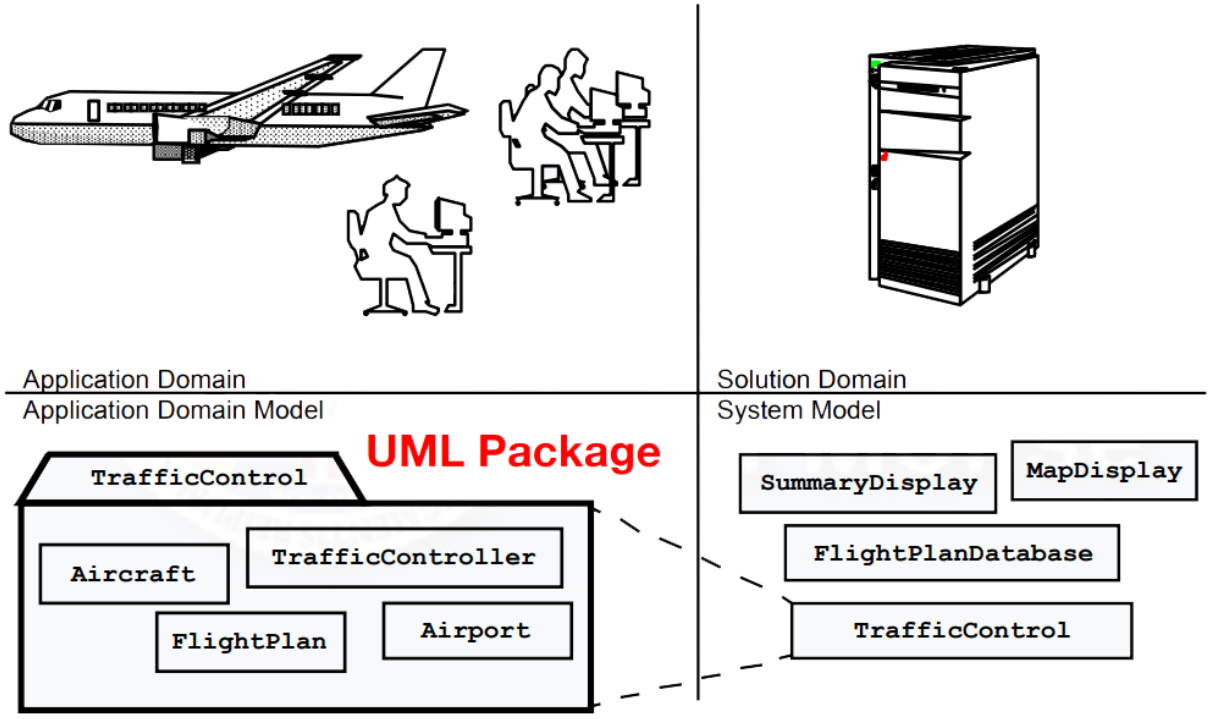



Figura 3 - Esempio di modellazione Object Oriented

Dalla individuazione del dominio di applicazione e del dominio di soluzione di definiscono i modelli di applicazione e di sistema.

**Per la modellazione si utilizza UML (Unified Modeling Language)**. UML è uno standard per modellare software object-oriented. È il risultato della convergenza di notazioni provenienti da tre dei principali metodi object-oriented: OMT (James Rumbaugh), OOSE (Ivar Jacobson), Booch (Grady Booch).

UML definisce diversi diagrammi:

- **Use case diagrams**: descrivono il comportamento funzionale del sistema dal punto di vista dell'utente.
- **Class diagrams**: descrivono la struttura statica del sistema: Oggetti, Attributi, Operazioni e Associazioni.

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

- **Sequence diagrams**: descrivono il comportamento dinamico tra gli attori ed il sistema e tra gli oggetti del sistema.
- **Statechart diagrams**: descrivono il comportamento dinamico di un singolo oggetto come una macchina a stati finiti
- **Activity diagrams**: modellano il comportamento dinamico del sistema, in particolare il workflow, es. flowchart.

UML utilizza diverse convenzioni. I rettangoli per rappresentare classi e istanze. Gli ovali che rappresentano funzioni e casi d'uso. Le istanze sono contraddistinte dal nome sottolineato. I tipi sono contraddistinti dal nome non sottolineato. Quasi tutti i diagrammi sono dei veri e propri grafi in cui i nodi sono le entità e gli archi sono le relazioni tra le entità.

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*